

End-to-End Synthesis: Value Elicitation Pipeline Redesign

Document 08 | March 2026 **Status:** Implementation specification synthesizing Research Notes 01-07
Depends on: All prior research documents in this series

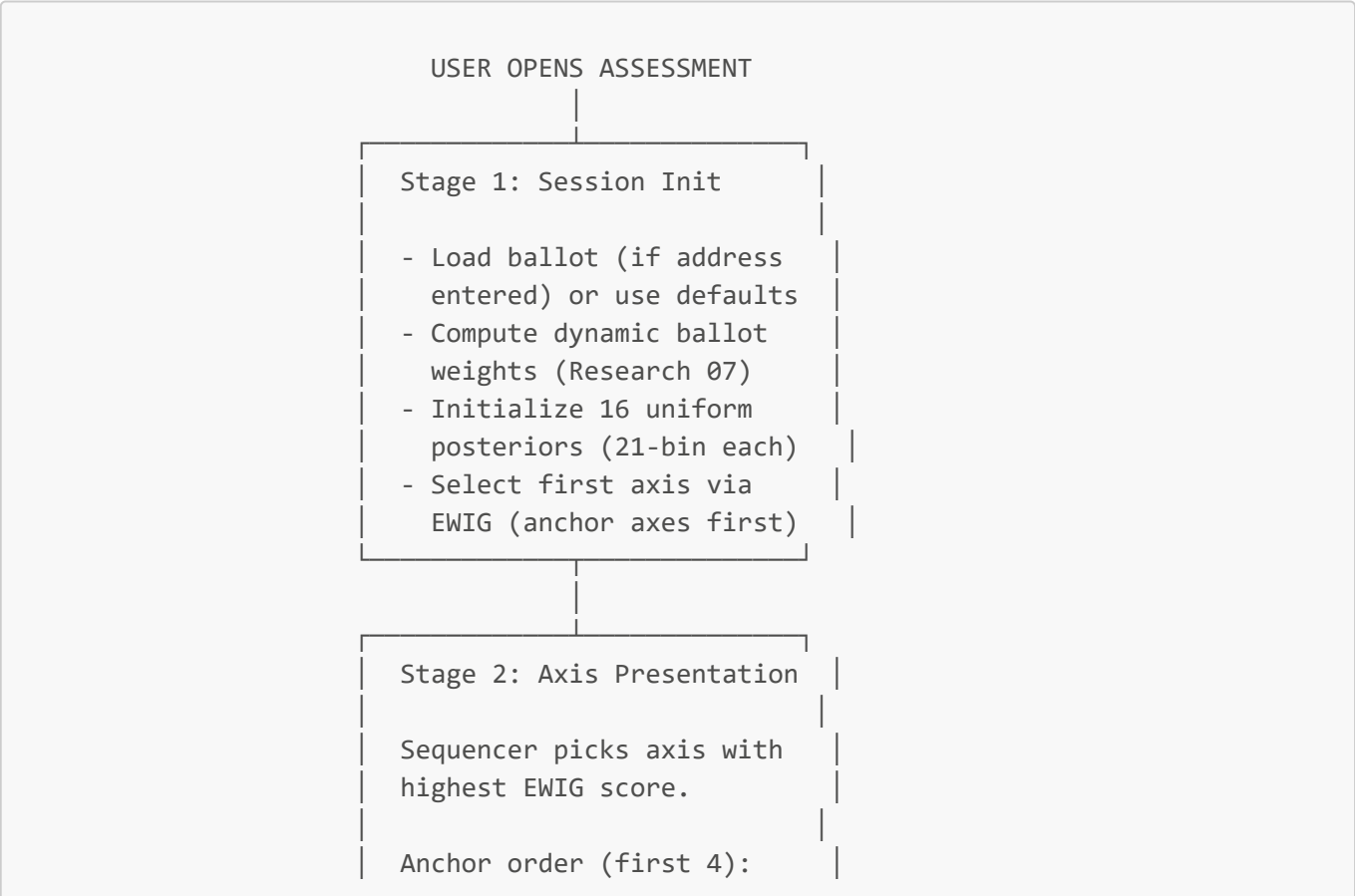
Table of Contents

- 1. [Architecture Overview](#)
- 2. [Three Paths Compared](#)
- 3. [Concrete Flow Design](#)
- 4. [Data Model](#)
- 5. [Spec Delta](#)
- 6. [Implementation Phases](#)
- 7. [Risk Register](#)
- 8. [Concrete Recommendations](#)

1. Architecture Overview

1.1 Complete Data Flow

The redesigned pipeline moves a user from entry to ballot match scores through six stages. Each stage has clear inputs, outputs, and decision points.



1. health_coverage_model
2. justice_firearms
3. climate_ambition
4. housing_supply_zoning

Show 5-option card UI with
"None of these fit" escape

STRUCTURED PATH
(user picks card)

NLP PATH
(escape hatch / voice)

Stage 3a: Structured Signal Processing

Card value → GRM-
informed posterior
update

Sigma from card
position + dwell time

Entropy-hybrid conf
($\alpha=0.35$)

Stage 3b: NLP Input

Speech-to-text (~500ms)

LLM extraction (all 16
axes, ~1500ms)

Signal classification:
primary / secondary /
spillover

Signal validation
(signalValidation.ts)

Entropy-hybrid conf
($\alpha=0.55$)

If secondary signals:
→ Confirmation card shown
→ User checks/unchecks

Stage 4: Posterior Update

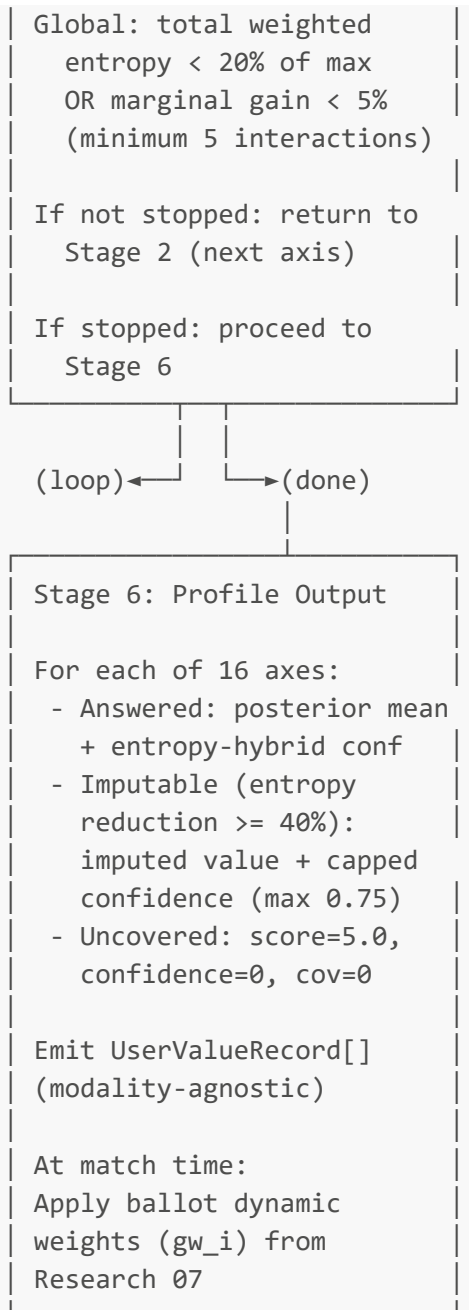
Bayesian update on the
answered axis (21-bin)

Cross-axis correlation
propagation (16x16 matrix)
Damped by $|\rho|$

Compute per-axis entropy

Stage 5: Stopping Check

Per-axis: entropy < 1.5
bits → axis resolved



1.2 Key Design Properties

Modality agnosticism. Both the structured and NLP paths produce the same output type (`UserValueRecord`). The matching formula does not know or care which path produced a given axis value. The only difference is in the `source_modality` field, which is informational.

Information-theoretic stopping. The system never asks a fixed number of questions. It measures how much it knows (via Shannon entropy of each axis posterior) and stops when the marginal value of asking one more question falls below a threshold. This naturally adapts to users who are expressive (stop sooner) vs. taciturn (ask more).

Cross-axis correlation exploitation. The 16x16 correlation matrix (Research 02, Section 4) encodes known relationships between US political attitudes. After answering `health_coverage_model`, the posterior on `econ_safetynet` (rho=0.55) shifts meaningfully. After 4-5 answers on anchor axes, 10+ axes have non-trivial posteriors without being directly asked.

Ballot-aware prioritization. When a ballot is loaded, the EWIG sequencer weights axes by their dynamic ballot relevance (Research 07). Axes where candidates disagree strongly on the user's specific ballot get asked first. If no ballot is loaded, all weights default to 1.0 and the sequencer optimizes for pure information gain.

2. Three Paths Compared

2.1 Comparison Table

Dimension	IRT-Enhanced Structured	Info-Theory NLP	Hybrid (Recommended)
Median questions to reach 80% axis coverage	8-10 (with cross-axis imputation)	6-8 turns (multi-axis extraction captures 2-3 axes per turn)	8-10 interactions total (mix of ~7 cards + ~2 voice), but voice interactions resolve 2-3 axes each
User effort per signal	Low: single tap per axis	Medium-High: formulate and speak/type a response	Variable: tap for clear-cut axes, speak for nuanced ones
Axes scorable per interaction	1 directly + ~0.5 via correlation propagation	1-3 per turn (primary + confirmed secondaries) + spillover	1-3 per interaction depending on modality chosen
Cold-start performance	Good: anchor axes (a=3) provide high information density from first question	Fair: first NLP turn provides little until topic is steered	Good: starts with structured anchor axes, voice available as escape hatch
Requires IRT calibration study?	Yes, for optimal item parameters. Heuristic a-estimates from Research 01 work as interim.	No (uses LLM extraction confidence)	No for MVP. IRT parameters improve item selection but are not required.
Implementation complexity	Medium: 21-bin posterior, EWIG computation, correlation matrix, stopping criterion	High: LLM extraction pipeline, signal validation, multi-axis classification, confirmation UI	High: all of the above, plus modality switching triggers and conflict resolution
Measurement precision at poles	High: strong-pole selections (a=3) produce tight posteriors (entropy ~1.3 bits)	Moderate: depends on LLM extraction quality; hedging reduces precision even for clear positions	High for poles (structured), moderate for nuanced positions (NLP with entropy confidence)

Dimension	IRT-Enhanced Structured	Info-Theory NLP	Hybrid (Recommended)
Handles cross-cutting positions	Poorly: 5-card format compresses nuance on axes like zoning, permitting	Well: free-form input can express conditional or context-dependent positions	Well: escape hatch specifically targets axes where cards are known to lose fidelity
Bias risk	Low: uniform prior, no ideological nudging	Medium: LLM extraction may encode systematic biases in confidence scoring (Failure Mode A from Research 06)	Medium: mitigated by entropy-hybrid confidence blending and signal validation
Offline/low-connectivity	Works fully offline after initial load	Requires network (LLM API + optional speech-to-text)	Graceful degradation: structured path works offline, NLP path requires network

2.2 Why Hybrid Wins

The hybrid approach captures 90%+ of the efficiency gains from both paths while avoiding their respective failure modes:

1. **Structured cards** are fast and precise for the 12-13 axes where most users have clear directional preferences. They require no LLM calls and produce deterministic, well-calibrated signals.
2. **NLP escape hatch** handles the 3-4 axes where cards compress too much nuance (housing zoning, climate permitting, policing accountability). A single rich voice response can resolve 2-3 housing axes simultaneously, saving 2 questions.
3. **The escape hatch framing ("none of these fit") is psychologically important.** It signals that the system respects the user's complexity rather than forcing them into boxes. This increases completion rates and trust.

3. Concrete Flow Design

Step 1: Entry

What the user sees:

A clean screen with a brief explanation: "Answer a few questions so we can match you with candidates on your ballot." Below: a progress indicator showing "About 8-10 questions" (adaptive, updates as the session progresses). If the user has previously entered an address, the ballot is pre-loaded and the progress estimate is more precise (based on ballot-weighted EWIG projection).

System state: `HybridAssessmentSession` is initialized. All 16 posteriors are uniform (21 bins, each at 1/21). If the user is returning, posteriors are initialized from their stored profile. Dynamic ballot weights are computed if a ballot is loaded; otherwise all weights are 1.0.

Step 2: Adaptive Card Selection

How the sequencer picks the next axis:

1. Compute EWIG for each unanswered axis. This involves simulating 5 hypothetical card selections per axis, computing the expected posterior entropy reduction across all 16 axes (weighted by ballot relevance), and selecting the axis with the highest expected gain.
2. Apply domain diversity bonus: if the last 2 questions were from the same domain, axes from other domains get a 10% EWIG bonus. This prevents monotony without sacrificing much information.
3. For the first 4 questions, the sequencer will typically select the anchor axes in order: `health_coverage_model`, `justice_firearms`, `climate_ambition`, `housing_supply_zoning`. These are chosen because they span the three latent dimensions of US political attitudes (economic left-right, social/cultural, environmental) and the cross-cutting housing dimension, maximizing cross-axis information propagation.

What the user sees: The axis topic is introduced with a plain-language question. Below: 5 selectable cards, each with a concise position label and a 2-sentence description. At the bottom: a "None of these fit -- tell us in your own words" link (the escape hatch). On NUANCED_AXES (`housing_supply_zoning`, `climate_permitting`, `justice_policing_accountability`), the escape hatch is visually enhanced with helper text.

Step 3: Card Interaction

Normal pick (structured path):

The user taps one of the 5 cards. The card animates to confirm selection. The system records:

- Selected value: one of [0.0, 2.5, 5.0, 7.5, 10.0]
- Dwell time: milliseconds from card display to selection
- Confidence assigned: 0.90 (poles), 0.80 (moderate), 0.70 (neutral)

The system computes the entropy-hybrid confidence:

```
hybrid_conf = 0.35 * entropy_confidence + 0.65 * heuristic_confidence
```

For a strong pole selection (dwell 3s): hybrid confidence ~0.831. For a confused neutral (dwell 800ms): hybrid confidence ~0.521.

The user immediately sees the next question. No LLM call is needed.

Trigger checks after selection:

- T2 (confused neutral): If value=5.0 AND dwell<2s, show a gentle "Want to tell us more?" chip.
- T3 (repeated neutrals): If 2+ consecutive neutrals, offer mode switch to conversation.

Step 4: Voice/Text Input (NLP Path)

When triggered: User taps "None of these fit" (T1), or responds to a T2/T3 prompt.

Recording UI:

- Cards remain visible but grayed out (maintains context).
- A text input area appears inline with a microphone button.
- Prompt: "Tell us what you think about [axis topic] in your own words."
- If voice: a pulsing ring animation during recording (already implemented as `VoiceButton.tsx`). Live partial transcription shown as the user speaks.
- If text: a standard text input with a character counter and send button.

Signal extraction (server-side, ~1500-2000ms):

1. Speech-to-text (if voice): ~500ms via the existing `/api/speech` endpoint.
2. LLM extraction (two-pass):
 - Pass 1: Conversational response generation (warm, guided by routing hint)
 - Pass 2: Value signal extraction against all 16 axes. The target axis gets the full position reference; the other 15 get abbreviated pole labels only.
3. Signal classification:
 - **Primary**: the asked axis. Always extracted. Full confidence.
 - **Secondary**: other axes where the user made an explicit, direct statement. Confidence discounted by 15%.
 - **Spillover**: axes implied by correlation graph (Research 03, Section 3.2). Confidence discounted by 40%. Never shown to user.
4. Signal validation (`signalValidation.ts`): Demotes suspicious neutral-with-high-confidence signals. Detects contradictions with prior signals.
5. Entropy-hybrid confidence computation ($\alpha=0.55$ for NLP path).

Step 5: Signal Confirmation

When secondary signals exist: The user sees a confirmation card:

We picked up a few things:

- ✓ [Housing zoning]: Build more with design standards
- [Transit]: Invest in public transit
- [Affordability]: Government help

Check the ones that sound right.

[Confirm & Continue]

← primary, locked

← secondary, unchecked

← secondary, unchecked

- Primary signal: pre-checked, cannot be unchecked.
- Secondary signals: unchecked by default. User must opt in.
- Confirmed secondaries are marked as "answered" and skip future questions for those axes.
- Unconfirmed secondaries are applied as soft evidence (update posterior but do not mark as answered).
- Spillover signals: never shown, always applied as soft evidence.

When no secondary signals: The primary signal is applied automatically and the next question appears.

Step 6: Progress & Stopping

How the user sees progress:

A progress bar showing "X of ~Y questions" where Y is the estimated total (recomputed after each interaction). Below the bar: a subtle "You can stop anytime" message. When the stopping criterion is met, the bar fills completely and shows "Profile complete!"

The estimated remaining count comes from the entropy summary: remaining weighted entropy divided by the average entropy reduction per interaction so far.

When the user can stop:

- The system stops automatically when global weighted entropy < 20% of maximum, or when marginal gain < 5% of remaining entropy (minimum 5 interactions).
- The user can also stop manually at any point after 5 interactions. The system will impute remaining axes from cross-axis correlations and clearly label them as "estimated."
- Typical completion: 8-10 interactions (7-8 cards + 1-2 voice), resolving 11-12 axes directly + 4-5 via imputation.

Step 7: Blueprint Output

Final profile display:

A summary screen showing all 16 axes organized by domain:

Your Civic Blueprint		
Economic Policy:		
Safety Net: Broader support (2.5)	[card]	●●●●○ high confidence
Public Investment: More investment (2.8)	[est.]	●●●○○ moderate confidence
School Choice: Traditional public (2.0)	[card]	●●●●○ high confidence
Tax Structure: Progressive (2.5)	[card]	●●●○○ moderate confidence
Healthcare:		
Coverage Model: Public option (2.5)	[card]	●●●●○ high confidence
Cost Control: Government oversight (3.1)	[est.]	●●○○○ low confidence
...		

Each axis shows:

- **Source badge:** [card], [voice], [est.] (estimated/imputed)
- **Confidence indicator:** filled dots proportional to entropy-hybrid confidence
- **Override option:** tap any axis to answer it directly (opens card or voice panel)

Imputed axes are visually distinct (dashed border, lighter color) and always show "Based on your other answers" with the source axes listed.

4. Data Model

4.1 UserValueRecord (Modality-Agnostic Output)

This is the core output type that both modalities produce and that the matching formula consumes.

```
interface UserValueRecord {
  axis_id: string;

  /** Position on the 0-10 scale */
  score: number;

  /** Normalized position on [-1, +1] for the matching formula */
  score_normalized: number;

  /** Entropy-hybrid confidence (0-1). This is what enters  $w_i$  as  $\text{conf}_i$ . */
  confidence: number;

  /** Coverage status for the matching formula */
  coverage_status: 'answered' | 'imputed' | 'uncovered';

  /** True if this value was derived from cross-axis correlation, not direct input */
  is_imputed: boolean;

  /** If imputed, which answered axes drove the estimate */
  imputation_source?: string[];

  // --- Provenance (informational, not used in matching) ---
  source_modality: 'structured' | 'nlp' | 'imputed';

  /** If structured: which card position was selected */
  card_position?: number;

  /** If NLP: the user's verbatim input */
  nlp_source_text?: string;

  /** If NLP: which signal strength classification */
  signal_strength?: 'primary' | 'secondary' | 'spillover';

  /** Timestamp of the record */
  recorded_at: number;
}
```

4.2 PosteriorState (Per-Axis Discretized Posterior)

The unified posterior representation uses 21 bins (0.0, 0.5, 1.0, ..., 10.0). Both structured and NLP signals update the same posterior via Bayesian multiplication.

```
interface PosteriorState {
  /** Probability mass over 21 bins. Must sum to ~1.0. */
  bins: number[]; // length 21
}
```

```

/** Shannon entropy in bits. H_max = log2(21) = 4.392 */
entropy: number;

/** Expected value: sum(bin_center * p_bin) */
mean: number;

/** Posterior standard deviation */
std: number;

/** Entropy-based confidence: 1 - (entropy / H_max) */
entropy_confidence: number;
}

/** Constants */
const N_BINS = 21;
const BIN_WIDTH = 0.5; // 10 / (N_BINS - 1)
const H_MAX = Math.log2(N_BINS); // 4.392 bits

```

Note on posterior resolution: Research 02 uses 5-point, Research 03 uses 11-bin, Research 06 uses 21-bin. The hybrid standardizes on 21-bin because it accommodates both structured selections (which map to bins 0, 5, 10, 15, 20) and continuous NLP signals. Conversion from 5-point to 21-bin is lossless.

4.3 SignalEvent (Individual Measurement)

Every piece of information the system receives is recorded as a SignalEvent for audit and debugging.

```

interface SignalEvent {
  /** Unique event ID */
  id: string;

  /** Which axis this signal targets */
  axis_id: string;

  /** The signal's direction on the 0-10 scale */
  direction: number;

  /** Raw confidence from the source (heuristic for structured, LLM for NLP) */
  raw_confidence: number;

  /** Entropy-based confidence after posterior construction */
  entropy_confidence: number;

  /** Final hybrid confidence (alpha-blended) */
  hybrid_confidence: number;

  /** Sigma used for the Gaussian posterior update */
  sigma: number;

  /** Which modality produced this signal */
  modality: 'structured' | 'nlp';
}

```

```

/** Signal strength classification (NLP only) */
strength: 'primary' | 'secondary' | 'spillover' | 'direct';

/** User's raw input (text for NLP, null for structured) */
source_text: string | null;

/** Whether the user confirmed this signal (secondary NLP only) */
user_confirmed: boolean;

/** Interaction number within the session */
interaction_number: number;

/** Timestamp */
timestamp: number;

/** Dwell time in ms (structured only) */
dwell_time_ms?: number;

/** Which trigger caused the modality switch, if any */
trigger_id?: string;
}

```

4.4 AssessmentSession (Full Session State)

```

interface AssessmentSession {
  session_id: string;

  // --- Global ---
  interaction_count: number;
  max_interactions: number; // default 20
  default_modality: 'structured' | 'nlp';

  // --- Per-axis state ---
  posteriors: Record<string, PosteriorState>;
  records: Record<string, UserValueRecord | null>;
  signal_history: Record<string, SignalEvent[]>;

  // --- Sequencing ---
  question_queue: string[]; // ordered by EWIG, recomputed after each
interaction
  answered_axes: Set<string>; // directly answered (structured or confirmed
NLP)
  skipped_axes: Set<string>; // user explicitly skipped
  imputed_axes: Set<string>; // resolved via cross-axis correlation

  // --- Modality tracking ---
  consecutive_neutrals: number;
  total_structured: number;
  total_nlp: number;
  modality_switches: Array<{

```

```

    from: string;
    to: string;
    trigger_id: string;
    interaction_number: number;
  }>;

  // --- Stopping ---
  session_entropy: number;
  estimated_remaining: number;
  ready_for_matching: boolean;
  stopping_reason?: string;

  // --- Ballot integration ---
  ballot_weights: Record<string, number>; // dynamic gw_i from Research 07
  ballot_loaded: boolean;

  // --- Correlation matrix ---
  correlation_matrix: number[][]; // 16x16, row-major

  // --- Pending confirmation ---
  pending_secondary_signals: SignalEvent[];
}

```

5. Spec Delta

5.1 Changes to Existing Files

src/server/data/civicAxes/spec.ts

What changes: Add IRT parameter stubs to each item. These are heuristic estimates from Research 01 that will be replaced with calibrated values after the IRT study.

```

// Add to each item object:
irt: {
  a: number; // discrimination (heuristic: 1, 2, or 3 per Research 01)
  b: [null, null, null, null]; // thresholds: null until calibration study
  info_at_zero: null; // null until calibration
}

```

Also add the 16th axis **econ_tax_structure** to the spec if not already present (the spec currently says 15 axes, but the domain definition includes 4 economic axes). Note: The spec file says "15 axes (3 per domain)" but the domain definition for **econ** already lists 4 axes including **econ_tax_structure**. This inconsistency should be resolved by updating the spec header to say 16 axes.

src/stores/userStore.ts

What changes:

- Add `posteriors: Record<string, number[]>` to the persisted state (16 keys, each a 21-element array). This is the per-axis posterior that survives across sessions for returning users.
- Add `assessmentSession: AssessmentSession | null` to track the active hybrid session.
- Add actions: `initializeAssessmentSession()`, `processStructuredSelection()`, `processNlpSignals()`, `processConfirmation()`, `finalizeAssessment()`.
- The existing `axisScores` remain for backward compatibility with the shrinkage scoring system. The new posteriors are a parallel representation that will eventually replace them.
- The existing `blueprintProfile` remains as the output format. The `finalizeAssessment()` action converts posteriors into `AxisProfile` objects, setting `value_0_10` from the posterior mean, `confidence_0_1` from the entropy-hybrid confidence, and `source` from the modality.

`src/stores/conversationStore.ts`

What changes: This store currently manages the domain-sequential warmup conversation. Under the hybrid flow:

- The `currentDomainIndex` and `domainTurnCount` fields become unnecessary (the entropy router replaces domain-sequential progression).
- Add `routerState: ConversationRouterState | null` from Research 03 to track per-axis posteriors and routing decisions during the NLP path.
- The store does NOT need to be replaced entirely. The existing ballot-item conversation logic (per-item messages, recommendation tracking, vote recording) remains unchanged. Only the warmup flow changes.
- Deprecate `advanceDomain()` action. Replace with `routeToNextAxis()` which uses the EWIG-based selection policy.

`src/types/blueprintProfile.ts`

What changes:

- Add optional `posterior?: number[]` field to `AxisProfile` to store the full 21-bin posterior alongside the point estimate. This enables richer UI displays (e.g., showing uncertainty ranges) without breaking existing consumers that only use `value_0_10`.
- Add optional `source_modality?: 'structured' | 'nlp' | 'imputed'` to `AxisProfile` for provenance tracking.
- No breaking changes. All new fields are optional.

`src/types/conversation.ts`

What changes:

- Update `ValueSignal` comment to say "16 civic axis IDs" (currently says 15).
- Add `signal_strength?: 'primary' | 'secondary' | 'spillover'` to `ValueSignal`.
- Add `entropy_confidence?: number` and `hybrid_confidence?: number` to `ValueSignal`.
- Add the `DOMAIN_AXES` entry for `econ_tax_structure` if not already present (it is present -- confirmed in the file).
- Add `ConversationRouterState` interface (from Research 03 Section 2.1) or import it from a new types file.

- Deprecate `currentDomainIndex` and `domainTurnCount` on `ConversationSession` (mark as `@deprecated`; do not remove for backward compatibility).

5.2 New Files Needed

File	Purpose	Research Source
<code>src/lib/entropyConfidence.ts</code>	Entropy-based confidence scoring (gaussianPosterior, shannonEntropy, entropyConfidenceStructured, entropyConfidenceNLP, bimodalPosterior)	Research 06, Section 3.4
<code>src/lib/posteriorEngine.ts</code>	21-bin posterior initialization, Bayesian update, cross-axis propagation, imputation	Research 02 Sections 2-3, Research 06
<code>src/lib/adaptiveSequencer.ts</code>	EWIG computation, next-axis selection with diversity bonus, stopping criterion evaluation	Research 02 Sections 5-7
<code>src/lib/correlationMatrix.ts</code>	The 16x16 cross-axis correlation matrix as a typed constant, plus helper functions	Research 02 Section 4
<code>src/lib/ballotWeighting.ts</code>	Candidate spread, ballot relevance, dynamic gw_i computation, dominance capping	Research 07
<code>src/lib/hybridSession.ts</code>	Session initialization, processStructuredSelection, processNlpResponse, processConfirmation, evaluateSessionStopping	Research 05 Sections 4, 7
<code>src/lib/modalityTriggers.ts</code>	Trigger evaluation logic (T1-T7), isConfusedNeutral, isVagueNlpResponse	Research 05 Section 2

File	Purpose	Research Source
<code>src/lib/multiAxisExtraction.ts</code>	Signal classification (primary/secondary/spillover), strength-based confidence discounting, confirmation card generation	Research 05 Section 5
<code>src/lib/conflictResolution.ts</code>	Merge rule for when both modalities produce records for the same axis, contradiction detection and resolution	Research 05 Section 6
<code>src/types/hybridAssessment.ts</code>	All TypeScript interfaces from Section 4 of this document	This document
<code>src/components/assessment/CardQuestion.tsx</code>	5-card selection UI with escape hatch, dwell time tracking, NUANCED_AXES enhanced escape	Research 05
<code>src/components/assessment/NlpPanel.tsx</code>	Inline voice/text input panel (appears below grayed cards)	Research 05
<code>src/components/assessment/ConfirmationCard.tsx</code>	Multi-axis signal confirmation UI	Research 05 Section 5.3
<code>src/components/assessment/AssessmentProgress.tsx</code>	Entropy-based progress bar with adaptive estimated remaining	Research 03 Section 6
<code>src/components/assessment/ProfileSummary.tsx</code>	Final profile display with per-axis source badges, confidence indicators, and override option	Research 05 Section 8
<code>src/app/api/assessment/extract/route.ts</code>	LLM extraction endpoint for multi-axis signal extraction from NLP input	Research 03 Section 7, Research 05 Section 5.4

5.3 Files That Can Be Deprecated

File	Reason	Timeline
<code>src/components/conversation/WarmupView.tsx</code>	Replaced by the hybrid assessment flow (CardQuestion + NlpPanel)	After Phase 1 ships
<code>src/components/conversation/ConversationProgress.tsx</code>	Replaced by AssessmentProgress.tsx (entropy-based)	After Phase 1 ships
Domain-sequential routing logic in <code>src/app/api/conversation/warmup/route.ts</code>	The warmup route can be refactored to use the entropy router instead of domain-sequential progression	After Phase 1 ships

Note: Do NOT delete these files. Mark them as deprecated and keep them until the hybrid flow is fully validated. The existing assessment flow (144-item questionnaire with shrinkage scoring) should remain as an alternative path.

6. Implementation Phases

Phase 0: Ship Without IRT Study (Pre-Calibration, 2-3 Weeks)

What ships: The entropy confidence system and basic adaptive sequencing, using heuristic IRT parameters.

Tasks:

1. Implement `src/lib/entropyConfidence.ts` (Research 06 Section 3.4). This is a pure function module with no external dependencies. Approximately 150 lines of TypeScript.
2. Implement `src/lib/posteriorEngine.ts` with 21-bin posteriors, Gaussian update, and cross-axis propagation. Use the heuristic correlation matrix from Research 02 Section 4.
3. Integrate entropy confidence into the existing signal validation pipeline (`src/server/services/signalValidation.ts`). The `confidence` field on `ValueSignal` becomes the hybrid value.
4. Add `entropyConfidenceNLP()` call after extraction in the warmup route. Store both heuristic and entropy confidence on the signal; compute hybrid.
5. Lower the NLP confirmation gate from 0.65 to 0.55 (or make it modality-dependent: 0.60 structured, 0.50 NLP).
6. Add dwell time tracking to the existing slider assessment UI (record time from question display to selection). This enables Failure Mode C disambiguation even before the card-based UI is built.

What this gives you: Fixes the most dangerous failure mode (B: confident-sounding vague answers get +110% weight inflation). Immediately improves match quality for all NLP users. No UX changes required.

IRT dependency: None. This phase uses no IRT parameters.

Phase 1: Hybrid Flow with Heuristic IRT (6-8 Weeks)

What ships: The full hybrid assessment experience -- card questions with voice escape hatch, adaptive sequencing, multi-axis extraction, entropy-based stopping.

Tasks:

1. Implement the full type system (`src/types/hybridAssessment.ts`).
2. Build the adaptive sequencer (`src/lib/adaptiveSequencer.ts`) with EWIG, stopping criterion, and imputation. Use heuristic `a` estimates from Research 01 (`a=1` for housing/transport/permitting, `a=2` for most axes, `a=3` for health_coverage, justice_firearms, climate_ambition, justice_policing).
3. Build the correlation matrix module (`src/lib/correlationMatrix.ts`).
4. Build the hybrid session manager (`src/lib/hybridSession.ts`).
5. Build the modality trigger system (`src/lib/modalityTriggers.ts`).
6. Build the multi-axis extraction system (`src/lib/multiAxisExtraction.ts`).
7. Build the conflict resolution module (`src/lib/conflictResolution.ts`).
8. Build the card question UI (`src/components/assessment/CardQuestion.tsx`) with 5 selectable cards, dwell time tracking, and the escape hatch. Note: position labels for each card should come from the civic axes spec (poleA/poleB labels).
9. Build the inline NLP panel (`src/components/assessment/NlpPanel1.tsx`). Reuse the existing `VoiceButton.tsx` component for recording.
10. Build the confirmation card (`src/components/assessment/ConfirmationCard.tsx`).
11. Build the progress bar (`src/components/assessment/AssessmentProgress.tsx`).
12. Build the profile summary (`src/components/assessment/ProfileSummary.tsx`).
13. Create the extraction API endpoint (`src/app/api/assessment/extract/route.ts`) with the modified Template A prompt that scans all 16 axes.
14. Wire everything together: new route or page for the hybrid assessment flow, session state management in the user store.

What this gives you: The "wow" experience. Users see 5 selectable cards, can tap one or say "none of these fit" and speak their mind. Voice responses resolve multiple axes at once. The system stops when it has enough information. Typical completion: 8-10 interactions instead of 16+ questions.

IRT dependency: Uses heuristic `a` estimates only. These determine the initial item ordering within each axis (higher `a` items are presented first) but do not affect the posterior update math. If the heuristic estimates are somewhat wrong, the system self-corrects after 2-3 answers via the correlation matrix.

Phase 2: Post-Calibration, Real IRT Parameters (4-6 Weeks After Study Completes)

What ships: Refined item parameters, validated correlation matrix, optimized stopping thresholds.

Tasks:

1. Run the IRT calibration study (Research 01 Section 5). N=1,500 respondents, matrix sampling design, 13-week timeline.
2. Fit the Graded Response Model and estimate `a` (discrimination) and `b` (threshold) parameters for all items.
3. Validate the 16x16 correlation matrix against empirical data. Update entries where discrepancy > 0.15.
4. Run CAT simulations to optimize stopping thresholds (target: match scores within 3 percentage points of full-information scores for 90% of users).
5. Update `src/server/data/civicAxes/spec.ts` with calibrated IRT parameters.

6. Update `src/lib/correlationMatrix.ts` with the empirical matrix.
7. Update stopping thresholds in `src/lib/adaptiveSequencer.ts` based on simulation results.
8. A/B test calibrated vs. heuristic parameters to verify improvement.

What this gives you: More precise adaptive routing (the sequencer knows exactly which items are most informative at the current posterior estimate), tighter stopping (reaches the accuracy target with fewer questions), and validated measurement properties (standard errors, classification accuracy).

Phase 3: Ballot-Weighted Matching (2-3 Weeks, Can Overlap with Phase 2)

What ships: Dynamic axis weights based on the user's specific ballot.

Tasks:

1. Implement `src/lib/ballotWeighting.ts` (Research 07). Candidate spread computation, ballot relevance normalization, dynamic weight formula, dominance capping.
2. Integrate dynamic weights into `computeCandidateMatches()` in `src/lib/ballotHelpers.ts`. The existing function takes axis weights; the change is computing those weights dynamically instead of using 1.0.
3. Pass dynamic weights to the adaptive sequencer as `ballotWeights`. The sequencer already accepts this parameter; we just need to provide real values instead of uniform.
4. Build the transparency UI: per-axis weight breakdown, weight explanation templates, ranking change disclosure.
5. Implement editorial override support for edge cases.

What this gives you: Match scores that emphasize the axes that actually differentiate candidates on the user's ballot. A voter whose ballot has a polarized Senate race on climate but an agreed-upon school board race on education will see climate weighted higher, producing more useful match scores.

Dependency: Requires candidate position data in the ballot pipeline (Phase 4 of `CLAUDE.md`). Can be partially deployed with relevance-only weighting (no spread) while candidate scoring catches up.

7. Risk Register

7.1 What Breaks if IRT Calibration Study Never Happens?

Impact: Moderate. The system works, but with reduced efficiency.

The heuristic `a` estimates from Research 01 are educated guesses based on published political science research (ANES discrimination parameters, Pew partisan gaps). They are likely correct in rank order: healthcare, firearms, and climate will have higher discrimination than housing zoning and climate permitting. But they may be off by 0.5-1.0 in absolute value.

Consequences without calibration:

- The adaptive sequencer may ask 1-2 extra questions before reaching the stopping criterion (asking about a mediocre item before a great one for a given axis).
- Cross-axis imputation may be slightly less accurate (the correlation matrix is theory-driven, not empirically validated for this specific instrument).

- The stopping thresholds (20% entropy floor, 5% marginal gain) are educated guesses. They may stop too early (producing 3-4% match score errors instead of the target 3%) or too late (asking 1-2 unnecessary questions).

Mitigation: The system is designed to be self-correcting. Even with wrong α estimates, after 4-5 answers the posteriors are dominated by observed data, not priors. The main cost is 1-2 extra questions vs. the calibrated optimum. This is acceptable for an MVP.

Recommendation: Ship Phase 1 without waiting for the calibration study. Run the study in parallel. Swap in real parameters in Phase 2.

7.2 Voice Recognition Failures

Fallback chain:

1. **Primary:** Browser Web Speech API for live transcription. If transcription quality is poor (WER > 30%), the system detects this via short/garbled transcripts.
2. **Secondary:** Show the transcribed text and ask "Did we get that right?" with an option to re-record or type instead.
3. **Tertiary:** Trigger T6 (vague NLP response) which shows the cards: "To make sure we capture your view, could you pick the card closest to what you described?"
4. **Final:** The user can always switch back to cards (T5). Voice is an escape hatch, not the only path.

The system should never get stuck in a state where the user has no way to proceed. Every NLP failure path loops back to the structured card UI within one interaction.

7.3 Under-Covered Axes (housing_supply_zoning, housing_transport_priority, climate_permitting, econ_tax_structure)

These 4 axes have the weakest validated instrument coverage (Research 01 Section 3). Housing zoning and transport have $\alpha=1$ (low discrimination). Climate permitting has $\alpha=1$. Tax structure has $\alpha=2$ but weak coverage of the flat/consumption tax pole.

Consequences:

- Items for these axes may not discriminate well. A user who is moderately pro-zoning-reform and one who is strongly pro-zoning-reform may give similar responses because the items are not sharp enough to distinguish them.
- Cross-axis imputation from other axes is also weak because these axes have low correlations with the main political dimensions (housing zoning: $r < 0.15$ with most non-housing axes).

Mitigation strategies:

1. **housing_supply_zoning** is an anchor axis (the 4th one), ensuring it is always directly asked. Direct measurement compensates for low discrimination.
2. These axes benefit most from the NLP escape hatch. The voice path can capture nuanced positions that the 5-card format compresses. Research 05 designates **housing_supply_zoning**, **climate_permitting**, and **justice_policing_accountability** as NUANCED_AXES with enhanced escape hatch prominence.

3. The 23 gap items proposed in Research 01 Appendix A specifically target these under-covered axes. Including them in the calibration study will improve discrimination estimates.
4. For `econ_tax_structure`, the existing 6 items plus the one proposed gap item ("A national sales tax...") provide adequate coverage of the progressive-vs-flat spectrum. The main risk is low engagement with the flat-tax concept among respondents unfamiliar with it.

7.4 Wrong Cross-Axis Correlations

Impact: Moderate for imputed axes, negligible for directly answered axes.

If the empirical correlation between, say, `climate_ambition` and `housing_transport_priority` is 0.15 instead of the theoretical 0.40, the system will over-propagate information from climate to transport. An aggressive climate activist who drives a truck would have their transport position incorrectly imputed as pro-transit.

Detection: The system tracks `imputation_source` on every imputed axis. If match results look wrong, the user can inspect which axes are imputed and override them.

Mitigation:

1. The propagation is damped by $|\rho|$. Even at $\rho=0.40$, the damping factor limits the shift magnitude.
2. Imputed confidence is capped at 0.75 (Research 02 Section 7), ensuring imputed axes always have lower effective weight than directly answered axes.
3. The profile summary clearly labels imputed axes and offers one-tap override.
4. Phase 2 replaces the theoretical matrix with empirically validated correlations. The design specifically anticipates matrix updates (Research 02 Section 9.1).

Worst case: A few users see imputed values they disagree with. They tap to override. This is recoverable and does not permanently damage the profile.

7.5 Scale Discrepancy: Euclidean Distance vs. Weighted Agreement

The current matching uses Euclidean distance on 0-10 scale:

$$A_i = 1 - |U_i - C_i| / 10 \quad (0 = \text{max disagreement}, 1 = \text{perfect alignment})$$

The spec from the research documents uses weighted agreement on [-1, +1]:

$$A_i = 1 - |U_i - C_i| / 2 \quad (\text{same formula, different scale})$$

These are mathematically equivalent when the scale mapping is consistent. The conversion is: `score_normalized = (score_0_10 - 5) / 5`, which maps [0, 10] to [-1, +1]. The research documents define this mapping explicitly in the `UserValueRecord` interface (`score_normalized`).

Action needed: Ensure all components use consistent scale conventions. The posteriors operate on [0, 10] internally (matching the 21-bin representation). The matching formula should receive normalized [-1, +1] values. The conversion happens at the boundary between the assessment system and the matching system.

This discrepancy is a naming/convention issue, not a mathematical issue. It does not need to be "resolved" -- just documented and handled consistently at the interface boundary.

8. Concrete Recommendations

8.1 Build Order Priority

Priority	Task	Effort	Impact	Dependency
P0	<code>src/lib/entropyConfidence.ts</code>	1 day	Fixes Failure Mode B (the most dangerous confidence error)	None
P1	Integrate entropy confidence into signal validation pipeline	1 day	Propagates fix into match scores	P0
P2	<code>src/lib/posteriorEngine.ts</code> + <code>src/lib/correlationMatrix.ts</code>	3 days	Foundation for all adaptive behavior	None
P3	<code>src/lib/adaptiveSequencer.ts</code>	3 days	Core sequencing intelligence	P2
P4	<code>src/types/hybridAssessment.ts</code>	1 day	Type foundation	None
P5	Card question UI (<code>CardQuestion.tsx</code>) with dwell tracking + escape hatch	4 days	Primary user-facing component	P4
P6	Multi-axis LLM extraction endpoint	3 days	Enables NLP path	P2
P7	<code>NlpPanel.tsx</code> + <code>ConfirmationCard.tsx</code>	3 days	NLP UI components	P5, P6
P8	<code>src/lib/hybridSession.ts</code> (session orchestration)	3 days	Ties everything together	P2, P3, P6
P9	Profile summary + progress bar	2 days	Assessment completion UX	P8
P10	<code>src/lib/ballotWeighting.ts</code>	2 days	Ballot-aware matching	P2
P11	Transparency UI (per-axis weight breakdown)	3 days	Trust and explainability	P10

8.2 What to Prototype First for Demo

The highest-impact demo artifact is the card-to-voice transition on a nuanced axis.

Build this specific flow first:

1. Show the `housing_supply_zoning` card question (5 options about zoning).
2. User taps "None of these fit."

3. The cards gray out, and the NLP panel slides in from below.
4. User speaks: "I want more housing built, especially near transit stops..."
5. After 1.5 seconds of processing, a confirmation card appears showing 3 detected signals (zoning, transit, affordability).
6. User checks "transit", confirms.
7. The progress bar jumps forward (3 axes resolved in 1 interaction).
8. The next card question appears (the sequencer skipped transit and affordability).

This flow demonstrates:

- The escape hatch UX (cards are not a dead end)
- Multi-axis extraction (one voice input resolves 3 axes)
- User confirmation (the system does not put words in your mouth)
- Adaptive sequencing (the system is smarter after your voice response)
- Progress acceleration (visible benefit of being expressive)

Implementation shortcut for demo: Hard-code the extraction results for a canned voice input. This lets you demo the UI flow without the full LLM extraction pipeline being production-ready.

8.3 Maximum "Wow" for Minimum Implementation Effort

Tier 1 (1-2 days, pure backend): Entropy confidence fix.

Implement `entropyConfidence.ts` and integrate it into the signal validation pipeline. This produces no visible UX change but immediately improves match quality for all NLP users. It is the highest-leverage single change.

Tier 2 (3-4 days, visible UX): Card-based assessment with adaptive ordering.

Replace the current 144-item slider assessment with a 16-card assessment (one card per axis, 5 options each). Order cards by the EWIG sequencer with the heuristic correlation matrix. Stop when entropy is low enough. This gives users a much shorter, smarter assessment experience without any voice/NLP complexity.

Tier 3 (1-2 weeks, the "wow" moment): Add the voice escape hatch.

Add the "None of these fit" button to cards, the inline NLP panel, and the multi-axis confirmation card. This is the feature that makes people say "this is different from every other quiz I've taken."

Tier 4 (2-3 weeks, deep trust): Ballot-weighted matching with transparency.

Show users exactly why their match scores look the way they do: which axes matter most for their specific ballot, where candidates disagree, and how confident the system is in each axis. This turns match scores from opaque numbers into explainable decisions.

8.4 What NOT to Build Yet

- **Full IRT scoring (EAP estimation):** The shrinkage scoring works fine. IRT parameters are useful for item ordering, not for replacing the scoring algorithm. Defer to Phase 2 or later.
- **Adaptive max_turns based on ballot complexity:** The fixed max of 20 interactions is generous enough. Optimize later with data.

- **User engagement signals (response time feeding into fatigue model):** Dwell time for neutral disambiguation is valuable (Phase 0). Full engagement modeling is premature.
- **Spillover graph learning from user data:** The static graph from Research 03 is good enough for launch. Learn from data after accumulating sessions.
- **Joint multi-axis posterior:** The per-axis posterior with pairwise correlation propagation is a principled approximation. A full 16-dimensional joint posterior is computationally expensive and premature for the current system state.

Appendix A: Axis Reference (All 16)

#	Axis ID	Domain	Heuristic a	Anchor?
0	econ_safetynet	Economic	2	No
1	econ_investment	Economic	2	No
2	econ_school_choice	Economic	2	No
3	econ_tax_structure	Economic	2	No
4	health_coverage_model	Healthcare	3	Yes (1st)
5	health_cost_control	Healthcare	2	No
6	health_public_health	Healthcare	2	No
7	housing_supply_zoning	Housing	1	Yes (4th)
8	housing_affordability_tools	Housing	2	No
9	housing_transport_priority	Housing	1	No
10	justice_policing_accountability	Justice	3	No
11	justice_sentencing_goals	Justice	2	No
12	justice_firearms	Justice	3	Yes (2nd)
13	climate_ambition	Climate	3	Yes (3rd)
14	climate_energy_portfolio	Climate	2	No
15	climate_permitting	Climate	1	No

Appendix B: Constants Reference

```
// === Posterior ===
const N_BINS = 21;
const H_MAX = Math.log2(N_BINS); // 4.392 bits

// === Entropy-hybrid blending ===
const ALPHA_STRUCTURED = 0.35;
const ALPHA_NLP = 0.55;
```

```

// === Stopping ===
const MIN_INTERACTIONS = 5;
const MAX_INTERACTIONS = 20;
const ENTROPY_FLOOR_FRACTION = 0.20; // stop if < 20% entropy
remains
const MARGINAL_GAIN_THRESHOLD = 0.05; // stop if best next gain <
5%
const PER_AXIS_ENTROPY_THRESHOLD = 1.5; // bits; axis resolved

// === Imputation ===
const IMPUTATION_ENTROPY_REDUCTION_THRESHOLD = 0.40; // minimum 40% entropy
reduction to impute
const IMPUTATION_CONFIDENCE_CAP = 0.75; // imputed axes capped below
direct answers

// === Confirmation gates ===
const STRUCTURED_CONFIRMATION_GATE = 0.60;
const NLP_CONFIRMATION_GATE = 0.50;

// === Signal strength discounts ===
const SECONDARY_DISCOUNT = 0.85; // 15% discount
const SPILLOVER_DISCOUNT = 0.60; // 40% discount

// === Modality triggers ===
const CONFUSED_NEUTRAL_DWELL_MS = 2000;
const CONSECUTIVE_NEUTRAL_THRESHOLD = 2;
const NLP_CONVERGENCE_TURNS = 3;
const NLP_CONVERGENCE_CONFIDENCE = 0.50;

// === Ballot weighting ===
const WEIGHT_BASE = 1.0;
const WEIGHT_SPREAD_FLOOR = 0.15;
const WEIGHT_RELEVANCE_FLOOR = 0.20;
const GW_MIN = 0.10;
const GW_MAX = 2.00;
const DOMINANCE_CAP = 0.35;
const MIN_ACTIVE_AXES = 3;

// === Contradiction ===
const CONTRADICTION_SCORE_GAP = 3.0; // on 0-10 scale

// === EWIG diversity ===
const DOMAIN_DIVERSITY_BONUS = 1.10; // 10% EWIG bonus for domain
switch

```

Appendix C: Cross-Document Dependency Map

Research 01 (IRT Calibration)

- └ provides: heuristic a-estimates, anchor axis selection, gap items
- └ consumed by: Research 02 (item ordering in sequencer)
 - Phase 2 (real parameters after study)

Research 02 (Adaptive Sequencing)

- └ provides: EWIG algorithm, posterior update, correlation matrix, stopping criterion, imputation rules
- └ consumed by: Research 05 (hybrid session orchestration)
This synthesis (Stage 2-5 of the pipeline)

Research 03 (Conversation Router)

- └ provides: entropy-based routing, multi-axis extraction, spillover graph, session stopping policy
- └ consumed by: Research 05 (NLP path within hybrid flow)
This synthesis (Stage 3b-4 of the pipeline)

Research 05 (Hybrid Flow)

- └ provides: modality switching triggers, unified session state, confirmation flow, conflict resolution, end-to-end walkthrough
- └ consumed by: This synthesis (Section 3 concrete flow, Section 5 spec delta)
- └ depends on: Research 02, 03, 06

Research 06 (Entropy Confidence)

- └ provides: entropy confidence formula, hybrid alpha blending, failure mode analysis, calibration comparison
- └ consumed by: Research 05 (confidence computation in both paths)
Research 07 (conf_i in the weight formula)
This synthesis (all stages)
- └ depends on: none (foundational)

Research 07 (Ballot Weighting)

- └ provides: dynamic gw_i computation, candidate spread, ballot relevance, adversarial constraints, transparency
- └ consumed by: Research 02 (ballotWeights input to EWIG)
This synthesis (Stage 1 init, Stage 6 matching)
- └ depends on: Research 06 (conf_i interaction)